

INDEX BUILD TIMING

Objective:

Compare the total latency when

(a) an index is created before data insertion versus

(b) an index is created after data insertion.

(Note:

While creating the table, do not define any primary key constraint, as it may affect the experimental results.)

Scenario (a) Index before Data Loading

The screenshot shows a PostgreSQL IDE interface with a query window and a query plan output. The query window contains the following SQL code:

```
25 FROM lineitem
26 WHERE l_quantity <= 10;
27
28 CREATE TABLE lineitem_a (LIKE lineitem INCLUDING ALL);
29
30 CREATE INDEX idx_lineitem_a_quantity
31 ON lineitem_a (l_quantity);
32
33 EXPLAIN ANALYZE
34 INSERT INTO lineitem_a
35 SELECT * FROM lineitem;
36
37
38
39
40
41
42
```

The query plan output shows the following steps:

Step	Operation	Cost	Time	Rows	Width	Loops
1	Insert on lineitem_a	(cost=0.00..1585237.92 rows=0 width=0)	(actual time=506945.469..506945.469 rows=0 loops=1)			
2	Buffers: shared hit=284209700 read=986125 dirtied=1141001 written=2008550					
3	-> Seq Scan on lineitem	(cost=0.00..1585237.92 rows=59988592 width=98)	(actual time=40.519..17465.238 rows=59986052.00 loops=1)			
4	Buffers: shared read=985352 written=5					
5	Planning:					
6	Buffers: shared hit=78 read=11 dirtied=4					
7	Planning Time: 479.547 ms					
8	Execution Time: 507110.807 ms					

Total rows: 8 Query complete 00:08:32.357

Index on l_quantity was created before loading data with the following results on query execution:

```
Insert on lineitem_a
(cost=0.00..1585237.92 rows=0 width=0)
(actual time=506945.469..506945.469 rows=0 loops=1)
```

As index structure existed before data insertion, every row updates the index.

506,945 ms = 507 seconds ~ 8.5 minutes.

Buffers:

```
shared hit=284,209,700
read=986,125
dirtied=1,141,001
written=2,008,550
```

The above results describe how the database interacted with your computer's memory (RAM) and disk during the insertion:

- `shared hit=284,209,700` (RAM USAGE) :
DB found the data it needed in the **shared_buffers** (RAM allocated to PostgreSQL) 284209700 times ~ 284 million times showing that we repeatedly read the index pages already sitting in RAM.
- `read=986,125` (DISK USAGE) :
This represents the number of times the database had to access the SSD because data required was not present on RAM - which is slower (secondary memory).
- `dirtied=1,141,001 & written=2,008,550` (MEMORY UPDATES AND PHYSICAL WRITES) :
Pages that were “dirtied” or updated were changed in memory and then updated in disk memory while page “writes” includes new data to the table and constant updates to 1_quantity indexing.

Scenario (b) : Indexing after loading data onto table

i) Measuring Data Insertion time:

```
Insert on lineitem_b
(cost=0.00..1585237.92 rows=0 width=0)
(actual time=317961.852..317961.852 rows=0.00 loops=1)
Planning Time: 0.060 ms
Execution Time: 317961.877 ms
```

Time taken to insert data (before indexing) = **317961 ms ~ 318 sec ~ 5.3 mins**

Buffers:

```
shared hit=62,019,512
read=985,352
dirtied=985,306
written=179,0577
```

We can observe that Part B had 222,190,188 memory interactions because the database didn't need to look up the B-Tree index for every single row.

dirtyed (985,306) and **read** (985,352) are almost identical here. This means for every page the database read from the original table, it modified exactly one page in the new table. This is "clean" work—no extra overhead.

```
-> Seq Scan on lineitem (cost=0.00..1585237.92 rows=59988592
width=98)
```

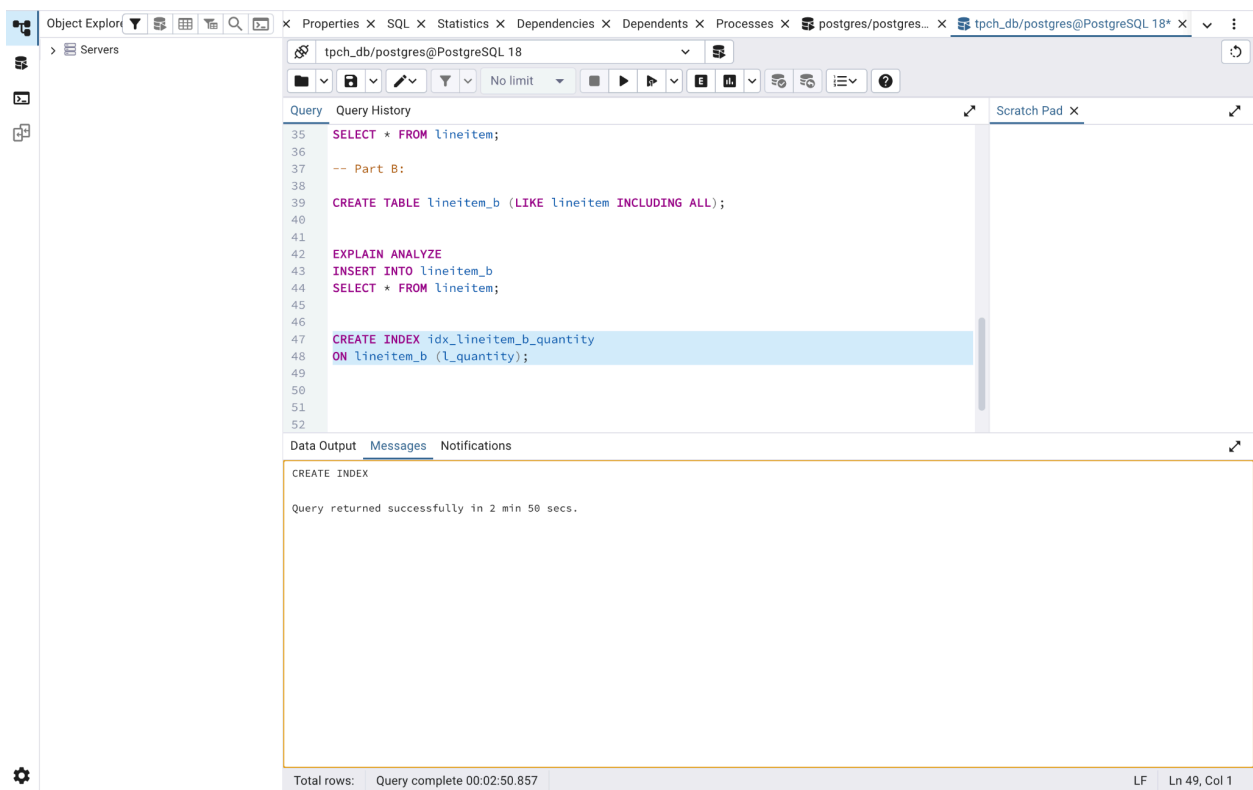
Postgres performed a "Sequential Scan." It reads the source data in a straight line - fastest way to move large amounts of data with minimal buffer hits when compared with scenario (a).

ii) Measuring time taken to Index

```
CREATE INDEX
```

```
Query returned successfully in 2 min 50 secs.
```

Total time taken in scenario (b) now becomes = **317,962 + 170,000 ≈ 487,962 ms**
≈ 8.13 minutes



The screenshot shows a PostgreSQL IDE interface. The main window displays a SQL query script with the following content:

```
35 SELECT * FROM lineitem;
36
37 -- Part B:
38
39 CREATE TABLE lineitem_b (LIKE lineitem INCLUDING ALL);
40
41
42 EXPLAIN ANALYZE
43 INSERT INTO lineitem_b
44 SELECT * FROM lineitem;
45
46
47 CREATE INDEX idx_lineitem_b_quantity
48 ON lineitem_b (l_quantity);
49
50
51
52
```

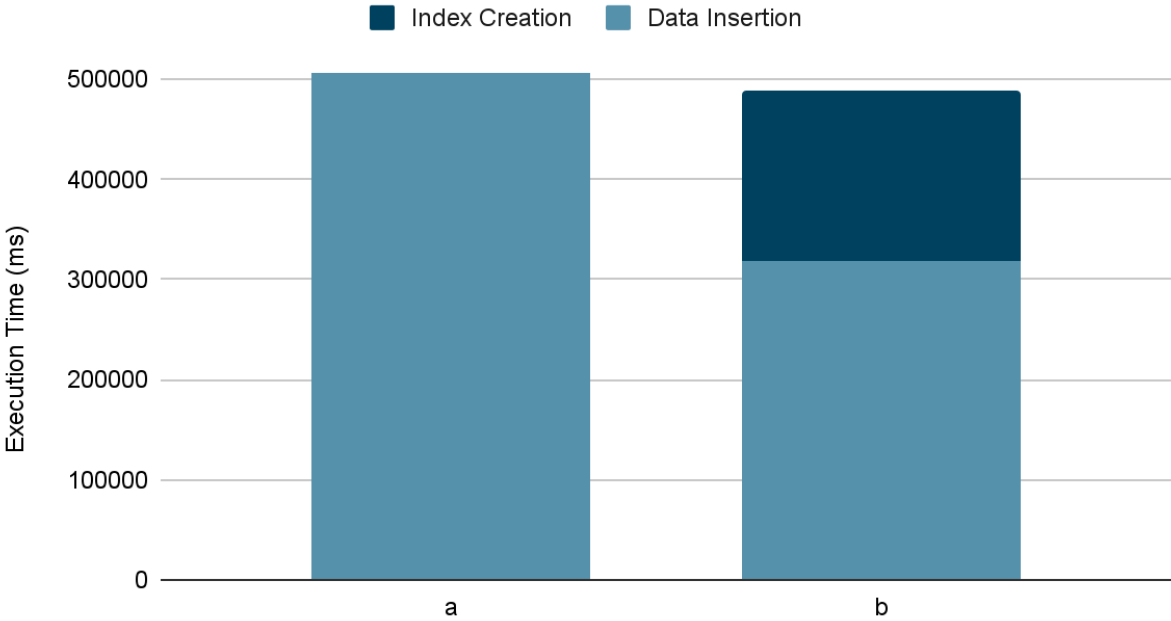
The query is executed, and the results are shown in the "Data Output" pane at the bottom. The output indicates that the index was created successfully in 2 minutes and 50 seconds.

Query complete 00:02:50.857

Phase	Scenario (a): Index Before	Scenario (b): Index After
Insert Data	506,945 ms (~8.45 min)	317,962 ms (~5.30 min)
Create Index	0 ms (Created at start)	170,000 ms (2.83 min)
Total Latency	506,945 ms	487,962 ms

RESULT : We observe that (b) runs 19 seconds faster.

Differences in Latency



FINAL COMPARISONS :

Scenario (b) is more efficient because of the following:

1. **Reduced Maintenance Overhead:** In Scenario (a), the database has to update the B-Tree for every row inserted involving complex logic to keep the tree balanced while handling 60 million rows.

2. **Bulk Sorting:**

In Scenario (b), when **CREATE INDEX** is executed on an already loaded table, PostgreSQL employs a bulk index build algorithm that :

- Reads table data sequentially
- Sorts index keys in large batches
- Constructs index pages in an ordered, linear fashion

-> more efficient than incremental index updates performed during individual inserts.

3. **Buffer Management:** **shared hit** count was much lower in Scenario (b) (62M vs 284M). This proves that Scenario (b) put significantly less strain on the system memory and CPU fewer repeated buffer accesses.

The time taken to execute (a) is high, for every row we:

- Write row onto table
- Traverse B-Tree index to find where the **I_quantity** value belongs - updating the node and also rebalancing the tree if required.
- Process leads to high I/O and CPU overhead.

Experimental Setup:

Dataset used: TPC-H Database(10GB)

System Configuration:

- Operating System: macOS Ventura 13.7.8
- RAM: 2 GB (allocated to PostgreSQL)
- CPU: 4 cores
- Storage: SSD
- PostgreSQL Version: 18
- Database Type (pgTune): Data Warehouse

pgTune Parameters:

max_connections = 20
shared_buffers = 512MB
effective_cache_size = 1536MB
maintenance_work_mem = 256MB
checkpoint_completion_target = 0.9
wal_buffers = 16MB
default_statistics_target = 500
random_page_cost = 1.1
work_mem = 10922kB
huge_pages = off
min_wal_size = 4GB
max_wal_size = 16GB
max_worker_processes = 4
max_parallel_workers_per_gather = 2
max_parallel_workers
max_parallel_maintenance_workers = 2

```
If Python could not import the module virtualenvwrapper.hook_loader,
check that virtualenvwrapper has been installed for
VIRTUAL_ENVWRAPPER_PYTHON=/usr/local/bin/python3 and that PATH is
set properly.
created virtual environment CPython3.11.9.final.0-64 in 84ms
creator Python3MacOSBrewEnv(dest=/Users/manasvigampan/virtualenvs/manasvi, clear=False, no_vcs_ignore=False, global=False)
seder from pipdata.download=False, pip=bundle, setuptools=bundle, wheel=bundle, via=copy, app_data_dir=/Users/manasvigampan/Library/Application Support/virtualenv/
added seed packages: appropose==1.4, asttokens==3.0.0, boto3==1.28.62, botocore==1.31.62, cffi==1.17.0, debugpy==1.8.5, decorator==5.2.1, executing==2.2.0, ipykernel==6.29.5, ipython==9.4.0, ipython
__pygments__lexer==1.11.1, jedi==0.19.2, jmespath==1.0.1, jupyter_client==8.6.3, jupyter_core==6.8.1, matplotlib-inline==0.1.7, nest_asyncio==1.6.0, net_tools==0.1.2, packaging==25.0, pandas
==2.0.0, pip==23.3, platformdirs==3.0, prompt_toolkit==0.0.1, psutil==7.0.0, pyprocess==0.7.0, pure_eval==0.2.3, pygments==2.18.2, python_dateutil==2.8.2, pytime==2.0.0, s3transfer==0.7.0, setuptools
==80.9.0, six==1.16.0, stack_data==0.0.3, tornado==6.5.1, tracemalloc==3.2.0, traitlets==5.14.3, typing_extensions==4.14.1, urllib3==2.0.6, wcwidth==0.2.13, wheel==0.45.1
activators BashActivator,CShellActivator,FishActivator,HubActivator,PowerShellActivator,PythonActivator
/usr/local/opt/python3.12/bin/python3.12: Error while finding module specification for 'virtualenvwrapper.hook_loader' (ModuleNotFoundError: No module named 'virtualenvwrapper')
+ -- clear

+ - psql -U postgres
Password for user postgres:
psql (18.1)
Type "help" for help.

postgres=# SELECT name, setting, unit, source
postgres=# FROM pg_settings
postgres=# WHERE name IN (
postgres=# 'max_connections',
postgres=# 'shared_buffers',
postgres=# 'effective_cache_size',
postgres=# 'maintenance_work_mem',
postgres=# 'checkpoint_completion_target',
postgres=# 'wal_buffers',
postgres=# 'default_statistics_target',
postgres=# 'random_page_cost',
postgres=# 'work_mem',
postgres=# 'huge_pages',
postgres=# 'min_wal_size',
postgres=# 'max_wal_size',
postgres=# 'max_worker_processes',
postgres=# 'max_parallel_workers_per_gather',
postgres=# 'max_parallel_workers',
postgres=# 'max_parallel_maintenance_workers'
postgres=# )
postgres=# ORDER BY name;
      name      | setting | unit | source
-----+-----+-----+-----
checkpoint_completion_target | 0.9     |      | configuration file
default_statistics_target   | 500     |      | configuration file
effective_cache_size        | 156688  | kB   | configuration file
huge_pages                  | off     |      | configuration file
maintenance_work_mem        | 262144  | kB   | configuration file
max_connections             | 20      |      | configuration file
max_parallel_maintenance_workers | 2       |      | configuration file
max_parallel_workers        | 4       |      | configuration file
max_parallel_workers_per_gather | 2       |      | configuration file
max_wal_size                | 16384   | MB   | configuration file
max_worker_processes        | 4       |      | configuration file
min_wal_size                | 4096    | MB   | configuration file
random_page_cost            | 1.1     |      | configuration file
shared_buffers              | 85536   | kB   | configuration file
wal_buffers                  | 2648    | kB   | configuration file
work_mem                    | 10922   | kB   | configuration file
(16 rows)

postgres=#
```